HDOC 15-1

NAME: DARSHIL RAJ
DATE: 15/10/26
SAP ID: 590040196

Program to Reverse a Given Number

This document provides a clean, well-commented implementation for reversing an integer in C. The program takes an integer as input, extracts its digits one by one using modulo and division operators within a `while` loop, and constructs the reversed number.

1. C Implementation

Step 1: Understand the Logic
To reverse a number (e.g., converting `1234` to `4321`), we repeatedly extract the last digit using the modulus operator (`num % 10`) and append it to our accumulator `reversed = reversed * 10 + digit`. Then, we remove the last digit from the original number by integer division (`num /= 10`).

- **Initialization:** `reversed = 0`.
- **Condition:** `num != 0`, so the loop continues as long as there are digits left to process.
- **Update:** `num /= 10` after each extraction step.

Step 2: Define Variables and Data Types
Use `int` for the original number (`num`), temporary backup (`n`), remainder/digit (`remainder`), and reversed result (`reversed`).

Step 3: Take User Input
Prompt the user using `printf("Enter an integer: ");` and read using `scanf("%d", &num);`.

Step 4: Reverse the Number Using a while Loop

1. Store the input in a temporary variable for final display.
2. Extract `remainder = num % 10`.
3. Update `reversed = reversed * 10 + remainder`.
4. Reduce `num` by dividing by 10 until it reaches 0.

Complete C Code

```
/* * Q25 (User Inputs, Operations & Output) * Write a program to reverse a given number. */ #include <stdio.h> int main() { int num, originalNum, remainder, reversed = 0; // Step 1: Read the value of num printf("Enter an integer: "); scanf("%d", &num); originalNum = num; // Save original for output display // Step 2: Reverse the number using a while loop while (num != 0) { remainder = num % 10; reversed = reversed * 10 + remainder; num /= 10; } // Step 3: Print the reversed result printf("The reverse of %d is: %d\n", originalNum, reversed); return 0; }
```

Program Output (Sample Runs)

```
Run 1: $ gcc hdoc15_1.c -o hdoc15_1 $ ./hdoc15_1 Enter an integer: 12345 The reverse of 12345 is: 54321 Run 2: $ ./hdoc15_1 Enter an integer: 1020 The reverse of 1020 is: 201 Run 3: $ ./hdoc15_1 Enter an integer: -456 The reverse of -456 is: -654 Run 4: $ ./hdoc15_1 Enter an integer: 7 The reverse of 7 is: 7
```

Dry Run with Test Cases

Sample Test Case 1:
Input: num = 123
Calculation:

- Start: `reversed = 0`, `num = 123`.
- Pass 1: 123 ≠ 0 is TRUE. `remainder = 3`, `reversed = 3`, `num = 12`.
- Pass 2: 12 ≠ 0 is TRUE. `remainder = 2`, `reversed = 32`, `num = 1`.
- Pass 3: 1 ≠ 0 is TRUE. `remainder = 1`, `reversed = 321`, `num = 0`.
- Check: 0 ≠ 0 is FALSE, loop terminates.

Output: The reverse of 123 is: 321

Sample Test Case 2:
Input: num = 5
Calculation:
Pass 1: `remainder = 5`, `reversed = 5`, `num = 0`. Loop ends immediately after one pass.
Output: The reverse of 5 is: 5

Sample Test Case 3:
Input: num = 0
Calculation: Loop condition 0 ≠ 0 is FALSE initially. Loop does not execute, `reversed = 0`.
Output: The reverse of 0 is: 0